

Laborator 1 MFIS

Laborator: Iova Ana
ana.iova@fmi.unibuc.ro

Subiecte Laborator

Search based testing

- ❖ Utilizarea algoritmilor evolutivi pentru satisfacerea criteriilor formale de testare software folosind EvoSuite
- ❖ Generarea automată de teste și date de intrare folosind AVMf și criterii formale de acoperire
- ❖ Generarea automată de scenarii de testare pentru sisteme autonome folosind AmbieGen
- ❖ Search based testing pentru EFSM - folosind Algoritmi Genetici hibridi
- ❖ Search based testing pentru EFSM - folosind algoritmi genetici multi-obiectiv

Metode formale de testare si verificare folosind limbajul Haskell

- ★ Testarea property-based a modulelor software folosind QuickCheck
- ★ Definirea și verificarea formală a unui limbaj de programare miniatural folosind Haskell
- ★ Definirea și verificarea formală a unui limbaj miniatural pentru expresii aritmetice folosind Haskell

Metode Formale

- Utilizează modele matematice (CFG, automată, logici) pentru a analiza riguros sistemele software.
- Oferă criterii precise: *branch coverage*, *path coverage*, *data-flow criteria*, *invarianti*, *specificații formale*.
- Scopul: garantarea corectitudinii sau detectarea sistematică a defectelor.

Search-Based Software Testing

- Transformă generarea de teste într-o problemă de optimizare.
- Folosește algoritmi de căutare euristică (genetici, hill-climbing).
- Scopul: maximizarea unei funcții de fitness (ex.: branch coverage).

Legătura dintre ele

- ✓ SBST folosește criterii formale ca obiective pentru optimizare (ex.: acoperirea unui CFG extras formal din cod).
- ✓ CFG-ul, criteriile de acoperire și modelele structurale → provin direct din Metode Formale.
- ✓ Metodele formale definesc ce trebuie acoperit; SBST găsește input-urile care ating acele criterii.
- ✓ SBST este partea „practică” și automatizată care operationalizează teoria formală în generarea testelor.

Metodele formale stabilesc regulile.

SBST găsește automat soluțiile.

Metode formale + HASKELL

De ce Haskell?

- Limbaj pur funcțional → model matematic direct (λ -calcul).
- Absența efectelor secundare → *facilitează analiza formală*.
- Tipare puternice & inference → verificări statice obiective.

Legătura cu Metodele Formale

- QuickCheck → testare ghidată formal (proprietăți = specificații).
 - LiquidHaskell → verificare statică formală cu SMT.
 - Modelele funcționale → ușor de tradus în specificații, hoare logic, invariante.
 - Permite îmbinarea între **specificații matematice** și **testare automată**.
- 

Testare bazată pe proprietăți (Property-Based Testing)

QuickCheck (Haskell)

- Testele sunt definite ca *proprietăți matematice* ("pentru orice input valid...").
- Generare automată de input-uri random sau structurate.
- Descoperă rapid cazuri limită + bug-uri greu vizibile.

```
prop_reverse :: [Int] -> Bool
```

```
prop_reverse xs = reverse (reverse xs) == xs
```

Coerență matematică și raționament formal

- Funcțiile pure $\approx$ funcții matematice $\rightarrow$ pot fi verificate prin:
 - ✓ inducție structurală
 - ✓ rescriere (rewrite systems)
 - ✓ dovezi de terminare
- Haskell permite modele formale fidele asupra execuției.



SBST

Algoritmi evolutivi în testarea software

Ce sunt algoritmi evolutivi?

- Tehnici de optimizare inspirate de selecția naturală
- Operatori principali: selecție, crossover, mutație
- Folosiți când spațiul de căutare este foarte mare sau greu de explorat exhaustiv
- Sunt utilizați pentru a găsi testele care optimizează un scop formal (fitness)



De ce sunt necesari în testarea ghidată formal?

Provocări:

- Criteriile formale (branch, path, MC/DC) pot fi greu de satisfăcut manual
- Spațiul de input este vast → căutarea completă este imposibilă
- Constrângerile din cod pot ascunde ramuri greu accesibile

Soluție:

Algoritmii evolutivi explorează spațiul de input-uri inteligent, „evoluând” teste către acoperirea criteriilor formale.



Conexiunea cu Metodele Formale

- ✓ Criteriile formale (ex.: covering all branches) devin **obiective matematice** în funcția de fitness
- ✓ CFG, grafurile de dependențe sau mutații definesc **proprietăți formale** ce trebuie satisfăcute
- ✓ Algoritmii evolutivi produc **input-uri care satisfac aceste proprietăți**
- ✓ Formal = definește ce trebuie atins
- ✓ Evolutiv = găsește cum să fie atins



EvoSuite

- ❖ Generează o populație inițială de teste (aleator sau heuristici)
- ❖ Evaluează fiecare test cu o funcție de fitness:
 - acoperire de ramuri
 - distanță până la atingerea unui branch
 - mutații detectate
- ❖ Selectează cei mai buni cromozomi (test cases)
- ❖ Aplică crossover și mutație → obține urmași
- ❖ Repetă ciclul până se maximizează criteriile formale
- ❖ Output: suite optimizate de teste JUnit



Generarea automată de teste cu AVMf

Ce este AVMf?

- ❖ AVMf (Alternating Variable Method framework) este o tehnică de optimizare locală folosită în Search-Based Software Testing.
- ❖ Implementată în multe tool-uri moderne (inclusiv EvoSuite pentru unele componente ale fitness-ului).
- ❖ Explorează spațiul de input variind câte o variabilă pe rând, în ambele direcții, pentru a minimiza distanța față de un obiectiv formal.

AVMf este utilizat pentru a găsi input-uri care:

- ❖ satisfac condiții formale din cod
- ❖ ating ramuri greu accesibile
- ❖ De ce e eficient?
 - ✓ este determinist și rapid
 - ✓ se adaptează bine la funcții de fitness complexe
 - ✓ optimizează input-urile fără să necesite algoritmi genetici complecși

Criterii formale de acoperire

Criterii folosite ca obiective:

- Acoperire de instrucțiuni
- Acoperire de ramuri (branch coverage)
- Acoperire de condiții (condition coverage)
- Acoperire de căi (path coverage)
- Mutation score (detectarea mutantilor)

AVMf optimizează input-urile astfel încât acestea să satisfacă aceste criterii formal definite.



Cum funcționează AVMf (schematic)

1. Se selectează un input inițial.
2. Se alege o variabilă de intrare.
3. Se mărește / micșorează valoarea ei cu pași mici.
4. Dacă fitness-ul se îmbunătățește, continuă în aceeași direcție.
5. Dacă nu, schimbă direcția sau variabila.
6. Oprește căutarea când obiectivul formal este satisfăcut.



Generarea automată de scenarii de testare pentru sisteme autonome folosind AmbieGen

Ce este AmbieGen?

- ❖ Un generator automat de scenarii pentru testarea sistemelor autonome
- ❖ Creează medii complexe: trafic, pietoni, obstacole, vreme, configurații spațiale
- ❖ Poate folosi:
 - ✓ căutare euristică
 - ✓ optimizare evolutivă
 - ✓ criterii formale de acoperire

AmbieGen = generator de medii (environment generator) controlat formal



Ce tipuri de scenarii generează?

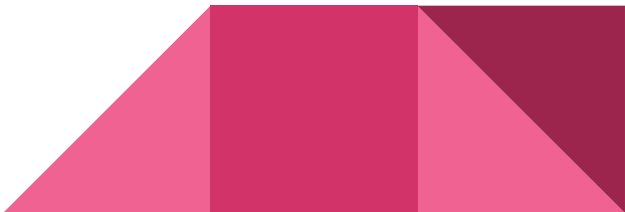
- ❖ Situații rare (rare events)
- ❖ Conflicte multiple simultane
- ❖ Scenarii cu risc ridicat: coliziuni, blocaje, trafic agresiv
- ❖ Configurații dinamice cu:
 - ✓ vehicule
 - ✓ pietoni
 - ✓ obiecte statice
 - ✓ obiecte imprevizibile
- ❖ Scopul: explorarea spațiului de mediu → descoperirea vulnerabilităților.

Criterii formale folosite în AmbieGen

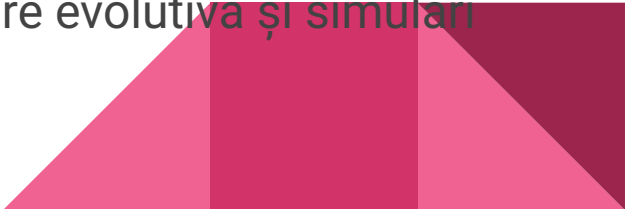
Criterii de acoperire:

- ❖ Acoperire spațială – distribuția pozițiilor în mediu
- ❖ Acoperire temporală – momente critice
- ❖ Acoperire comportamentală – tipare de trafic, acțiuni agenți
- ❖ Acoperire a situațiilor critice (near miss, coliziuni, braking)

Proprietăți formale:

- ❖ Invarianti de siguranță
 - ❖ LTL (Linear Temporal Logic) pentru secvențialitate
 - ❖ Constrângeri de traiectorie (trace constraints)
- 

Avantaje ale AmbieGen

- ✓ Scalabil pentru medii mari și complexe
 - ✓ Găsește scenarii rare dificil de obținut manual
 - ✓ Poate simula situații periculoase fără risc real
 - ✓ Compatibil cu testarea simulată și reală
 - ✓ Permite integrarea specificațiilor formale
 - ✓ Oferă o abordare formală și automată pentru generarea scenariilor de testare în sisteme autonome, combinând modele formale, optimizare evolutivă și simulări realiste.
- 

Testare bazată pe căutare (SBST) pentru EFSM

Ce este EFSM?

- ❖ O mașină cu stări finite extinsă care combină:
 - stări finite (ca în FSM clasic)
 - variabile de stare
 - gărzi (condiții logice)
 - acțiuni care pot modifica variabilele
- ❖ Poate modela sisteme reactive complexe (automotive, IoT, telecom, robotică).
- ❖ O tranziție are forma: **(stare curentă, eveniment, gardă) → (stare următoare, acțiune)**
- ❖ **Provocare:** spațiul de căutare al traseelor crește exponențial → testarea manuală dificilă.
- ❖ **Soluție:** Search-Based Testing (SBST)

EFSM

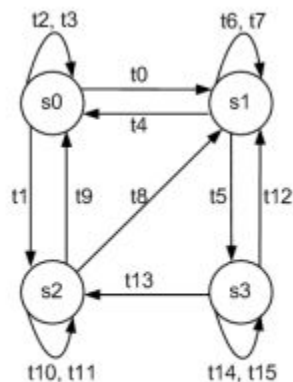


Fig. 1: Book EFSM model

t	$s_s \rightarrow s_e$	Input	Guards	Operations
t_0	$s_0 \rightarrow s_1$	$t_0(x)$	$x > 0$	$bId := x;$
t_1	$s_0 \rightarrow s_2$	$t_1(x)$	$x > 0$	$rId := x;$
t_2	$s_0 \rightarrow s_0$	$t_2(x)$	$x \leq 0$	<i>Nil</i>
t_3	$s_0 \rightarrow s_0$	$t_3(x)$	$x \leq 0$	<i>Nil</i>
t_4	$s_1 \rightarrow s_0$	$t_4(x)$	$x = bId$	$bId := 0;$
t_5	$s_1 \rightarrow s_3$	$t_5(x)$	$x > 0 \wedge x \neq bId$	$rId := x;$
t_6	$s_1 \rightarrow s_1$	$t_6(x)$	$x \neq bId$	<i>Nil</i>
t_7	$s_1 \rightarrow s_1$	$t_7(x)$	$x \leq 0 \vee x = bId$	<i>Nil</i>
t_8	$s_2 \rightarrow s_1$	$t_8(x)$	$x = rId$	$bId := x; rId := 0;$
t_9	$s_2 \rightarrow s_0$	$t_9(x)$	$x = rId$	$rId := 0;$
t_{10}	$s_2 \rightarrow s_2$	$t_{10}(x)$	$x \neq rId$	<i>Nil</i>
t_{11}	$s_2 \rightarrow s_2$	$t_{11}(x)$	$x \neq rId$	<i>Nil</i>
t_{12}	$s_3 \rightarrow s_1$	$t_{12}(x)$	$x = rId$	$rId := 0;$
t_{13}	$s_3 \rightarrow s_2$	$t_{13}(x)$	$x = bId$	$bId := 0;$
t_{14}	$s_3 \rightarrow s_3$	$t_{14}(x)$	$x \neq rId$	<i>Nil</i>
t_{15}	$s_3 \rightarrow s_3$	$t_{15}(x)$	$x \neq bId$	<i>Nil</i>

Tranziții EFSM

SBST - EFSM

- ❖ problemă de optimizare, și anume găsirea unei secvențe de intrări care:
 - satisface o condiție formală (ex. acoperă o tranziție)
 - atinge o stare specificată
 - maximizează un criteriu formal (coverage, mutation score)
- ❖ Modelul EFSM este folosit ca spațiu formal de căutare pentru algoritmi evolutivi.



Algoritmi genetici hibridi

Algoritmii genetici standard:

- ❖ Se pot bloca în minime locale
- ❖ Pot avea dificultăți în a explora condiții EFSM cu multe constrângeri

Hibridizarea (GA + altă tehnică) aduce îmbunătățiri:

- ✓ optimizare locală (hill-climbing, AVMf)
- ✓ reparare de trasee invalide (constraint solving)
- ✓ accelerarea convergenței

Algoritmi Genetici Multi-Obiectiv pt EFSM

EFSM includ:

- ❖ dimensiune structurală (stări, tranziții, bucle)
- ❖ dimensiune logică (gărzi, predicatii)
- ❖ dimensiune numerică (variabile, actualizări)

În testare trebuie optimizate mai multe obiective simultan, de exemplu:

- ❖ minimizarea nivelului de apropiere (approach level)
- ❖ maximizarea acoperirii tranzițiilor
- ❖ maximizarea diversității traseelor

MO-GA pot gestiona trade-off-uri între aceste obiective.



Algoritmi Genetici Multi-Obiectiv (MO-GA)

Cei mai utilizați algoritmi în testarea EFSM:

- ❖ NSGA-II (Nondominated Sorting Genetic Algorithm II)
 - produce un set de soluții Pareto-optime
- ❖ SPEA2 (Strength Pareto Evolutionary Algorithm)
- ❖ MO-EA/D (Evolutionary Algorithm based on Decomposition)

Avantaje:

- ✓ returnează un set de trasee, nu doar unul singur
 - ✓ evită soluțiile suboptimale prin dominance sorting
 - ✓ permite analizarea compromisurilor între obiective
- 